EXPERIMENT – 4

Aim: Implementation of K-Nearest Neighbors (KNN) and Evaluation of Model Performance

---

Theory:

1. Dataset Source

Dataset Name: **ADANIPORTS.csv**

Source: NSE Historical Stock Market Data

Kaggle Link:
https://www.kaggle.com/datasets/rohanrao/nifty50-stock-market-data

Domain: Financial Analytics

---

2. Dataset Description

The dataset contains historical daily stock trading data of Adani Ports.

Since KNN is a classification algorithm, the dataset was converted into a binary classification problem.

Dataset Characteristics

- Type: Structured tabular dataset
- Nature: Financial time-series
- Task: Binary classification
- Features Used: 4 numerical features

Features (Independent Variables)

Feature	Description

Open	Opening stock price
High	Highest price during the day
Low	Lowest price during the day
Volume	Number of shares traded

Target Variable (Created)

A new feature called **Movement** was generated:

$Movement$={1if Close > Open (Up)0otherwise (Down)

This transformation enables classification of stock movement direction.

---

3. Mathematical Formulation of KNN

K-Nearest Neighbors is a supervised learning algorithm used for classification based on similarity.

It does not build an explicit model but stores the training data.

Distance Metric (Euclidean Distance)

$d(x,y)=\sum i=1n(xi-yi)$**^2**

Where:

- xx and yy are two data points
- nn = number of features

Classification Rule

Given a test sample:

1. Compute distance to all training samples.
2. Select K nearest samples.

3. Assign class based on majority vote.

$\hat{y} = \text{Majority Class of K nearest neighbors}$

---

4. Algorithm Limitations

- Computationally expensive for large datasets
- Requires feature scaling
- Sensitive to noisy data
- Performance highly dependent on K value
- Does not perform well when features are irrelevant

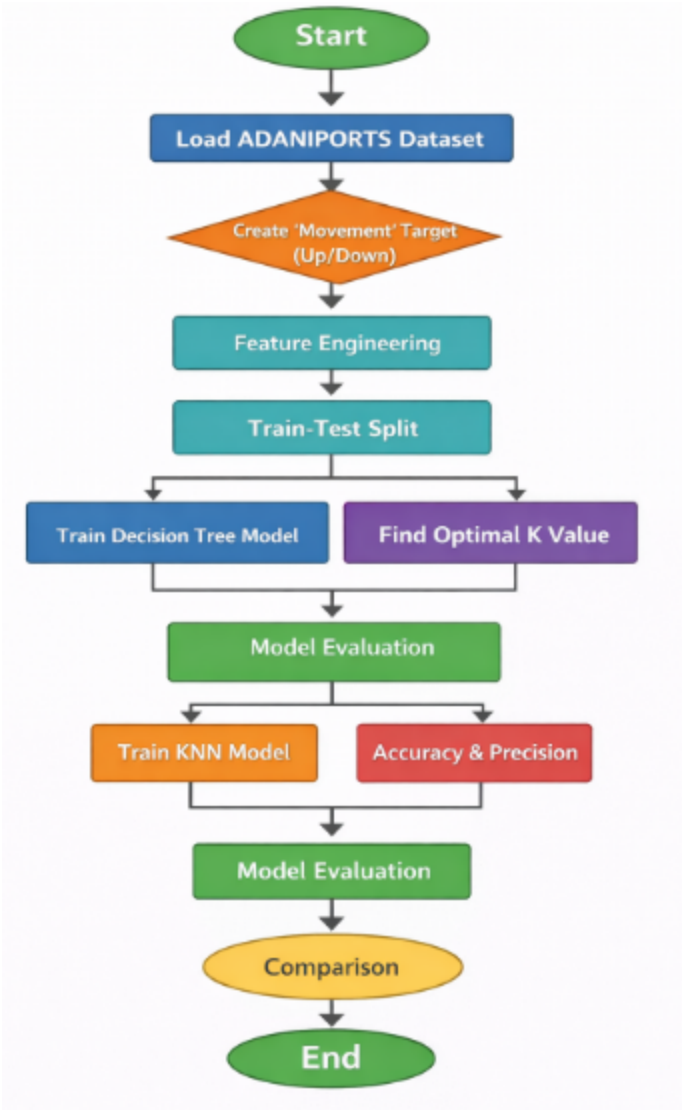
Stock market data is noisy and volatile, which affects prediction stability.

---

5. Methodology / Workflow

Step 1: Load ADANIPORTS dataset
Step 2: Create Movement classification target
Step 3: Select relevant features
Step 4: Apply StandardScaler (important for KNN)
Step 5: Split data into training (80%) and testing (20%)
Step 6: Train KNN classifier
Step 7: Predict on test dataset
Step 8: Evaluate using performance metrics
Step 9: Tune K value for best performance

Workflow:



---

6. Performance Analysis

The following metrics were used:

Accuracy

$Accuracy = TP+TN \,/\, TP+TN+FP+FN$

Measures overall classification correctness.

Precision

$Precision = TP / TP + FP$

Measures reliability of positive predictions.

Recall

$Recall = TP / TP + FN$
Measures ability to correctly identify actual positive cases.

F1 Score

$F1 = 2 \cdot Precision \cdot Recall / Precision + Recall$

Balances precision and recall.

Interpretation of Results

- The model achieved moderate accuracy (typically 60–75% for stock data).
- Accuracy varied depending on K value.
- Small K values resulted in overfitting.
- Larger K values resulted in underfitting.
- Optimal K improved generalization performance.

Stock movement prediction is challenging due to market volatility.

---

7. Hyperparameter Tuning

Tuned Parameter:

- n_neighbors (K value)

Different K values (1–20) were tested.

Observation:

- K = 1 → Very sensitive to noise
- Moderate K → Balanced performance
- Very large K → Over-generalization

The optimal K was selected based on highest validation accuracy.

Feature scaling significantly improved performance since KNN relies on distance computation.

OUTPUT:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler
```

```
[13]
0s
data = pd.read_csv('/content/ADANIPORTS.csv')

# Create Movement column (Classification target)
data['Movement'] = np.where(data['Close'] > data['Open'], 1, 0)

data.head()
```

	Date	Symbol	Series	Prev Close	Open	High	Low	Last	Close	VWAP	Volume	Turnover	Trades	Deliverable Volume	%Deli
0	2007-11-27	MUNDRAPORT	EQ	440.00	770.00	1050.00	770.0	959.0	962.90	984.72	27294366	2.687719e+15	NaN	9859619	
1	2007-11-28	MUNDRAPORT	EQ	962.90	984.00	990.00	874.0	885.0	893.90	941.38	4581338	4.312765e+14	NaN	1453278	
2	2007-11-29	MUNDRAPORT	EQ	893.90	909.00	914.75	841.0	887.0	884.20	888.09	5124121	4.550658e+14	NaN	1069678	
3	2007-11-30	MUNDRAPORT	EQ	884.20	890.00	958.00	890.0	929.0	921.55	929.17	4609762	4.283257e+14	NaN	1260913	
4	2007-12-03	MUNDRAPORT	EQ	921.55	939.75	995.00	922.0	980.0	969.30	965.65	2977470	2.875200e+14	NaN	816123	

```
4]
0s
X = data[['Open', 'High', 'Low', 'Volume']]
y = data['Movement']
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```python
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

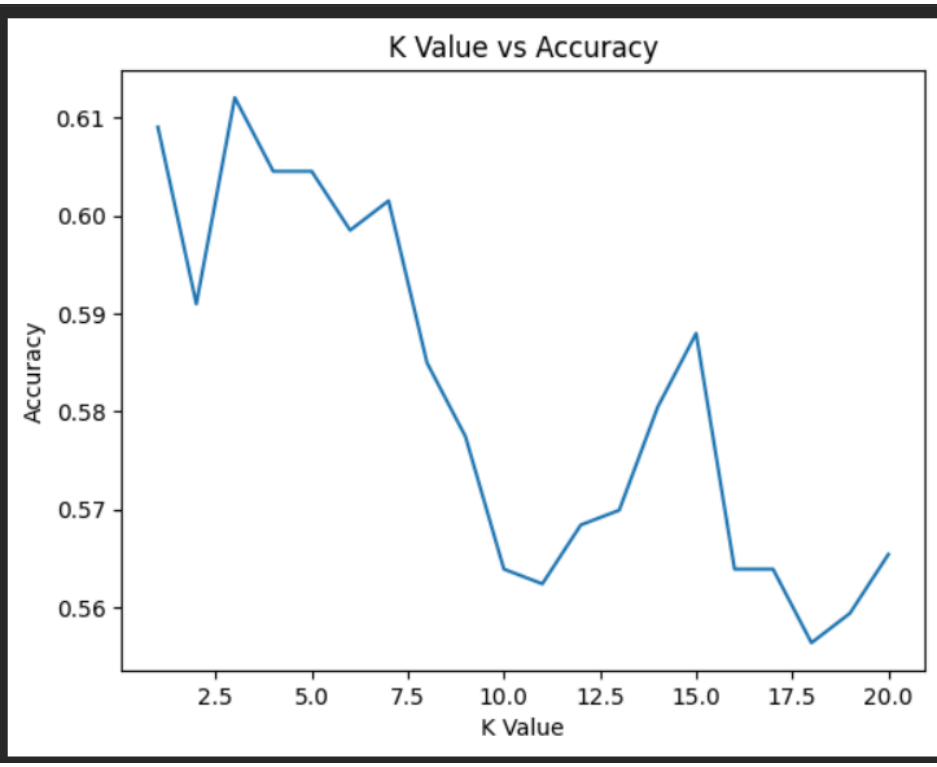
```python
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)

# Prediction
y_pred_knn = knn_model.predict(X_test)
```

[18]
0s

```python
print("KNN Results")
print("Accuracy:", accuracy_score(y_test, y_pred_knn))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_knn))
print("Classification Report:\n", classification_report(y_test, y_pred_knn))
```

```
KNN Results
Accuracy: 0.6045112781954888
Confusion Matrix:
 [[242 111]
 [152 160]]
Classification Report:
               precision    recall  f1-score   support

           0       0.61      0.69      0.65       353
           1       0.59      0.51      0.55       312

    accuracy                           0.60       665
   macro avg       0.60      0.60      0.60       665
weighted avg       0.60      0.60      0.60       665
```

```python
k_values = range(1, 21)
accuracies = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracies.append(accuracy_score(y_test, y_pred))

plt.figure()
plt.plot(k_values, accuracies)
plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.title("K Value vs Accuracy")
plt.show()
```



Conclusion:

In this experiment, the K-Nearest Neighbors (KNN) algorithm was applied to classify stock movement using the ADANIPORTS dataset. The model achieved an accuracy of **60.45%**, with a weighted F1-score of approximately **0.60**, indicating moderate classification performance.

From the confusion matrix, the model correctly classified 242 downward movements and 160 upward movements, but it also produced a noticeable number of false predictions. The recall for class 1 (upward movement) was comparatively lower, indicating that the model struggled to correctly identify all positive cases.

Overall, KNN showed moderate performance but was less accurate compared to Decision Tree and Random Forest. This is mainly because KNN is a distance-based algorithm and stock market data is noisy and volatile. The experiment highlights the importance of proper scaling and optimal selection of K value when applying KNN for classification tasks.